# Java Annotations

Metadata: Code that describes Code

CODEHIVE

# 1. What are Annotations?

Annotations are special markers or "metadata" notes you add to your Java source code. They always begin with the `@` symbol. They do not directly affect the execution of the program logic, but they provide vital information to the compiler, development tools, or the Java Runtime Environment.

```
// Example look of an annotation:
@Override
public String toString() {
    return "CodeHive Example";
}
```

## 2. Built-in Annotations

Java provides a set of standard annotations out of the box to help manage common development scenarios.

| Annotation | Development Use Case |
| --- | --- |
| `@Override` | Checks if a method truly overrides a parent method. |
| `@Deprecated` | Marks code as obsolete; warns users to avoid it. |
| `@SuppressWarnings` | Mutes specific compiler warnings in a scope. |
| `@SafeVarargs` | Suppresses warnings for generic variable arguments. |
| `@FunctionalInterface` | Ensures the interface has exactly one abstract method. |

## 3. The @Override Safeguard

The `@Override` annotation is a developer's best friend. It asks the compiler to verify that the method is actually overriding a method from a superclass.

```
class Animal {
    void makeSound() { System.out.println("..."); }
}

class Dog extends Animal {
    @Override
    void makeSound() {
        System.out.println("Woof!");
    }
}
```

## 4. Catching Typos

Without the annotation, a typo creates a new method instead of overriding the intended one. With the annotation, the compiler stops you immediately.

```
class Dog extends Animal {
    @Override
    void makesound() { // ERROR: Typo! Uppercase 'S' missing
        System.out.println("Woof!");
    }
}
```

**Result:** error: method does not override or implement a method from a supertype.

## 5. Managing Legacy: @Deprecated

As software evolves, some methods become outdated. `@Deprecated` signals that a method still exists but should no longer be used by developers.

```
public class API {
    @Deprecated
    public void oldLegacyLogin() {
        // Outdated logic
    }

    public void secureLogin() {
        // Modern logic
    }
}
```

## 6. Silencing the Compiler

Sometimes you write code that is intentionally complex or uses older styles (raw types). `@SuppressWarnings` keeps your build logs clean by hiding specific warnings.

```
@SuppressWarnings("unchecked")
public void handleLegacyData() {
    ArrayList list = new ArrayList(); // Raw type warning removed
    list.add("Data");
}
```

Use this sparingly; it is usually better to fix the warning than to hide it.

## 7. Functional Interfaces

The `@FunctionalInterface` annotation ensures that an interface is compatible with Lambda expressions by having exactly one abstract method.

```
@FunctionalInterface
public interface SimpleCalculator {
    int calculate(int x, int y);
    // Adding another method here would cause a compiler error
}
```

## 8. Annotation Retention

Annotations have different "lifespans" (Retention Policies):

- **SOURCE:** Digested by compiler; discarded afterwards.
- **CLASS:** Recorded in the .class file; ignored by the JVM.
- **RUNTIME:** Recorded in the .class file and accessible by the JVM via reflection.

## 9. Summary & Best Practices

- Always Use @Override: It documents intent and prevents silent bugs.
- Don't Over-Suppress: Only use @SuppressWarnings if the code cannot be refactored to be cleaner.
- Read the Javadoc: When you see @Deprecated, check the documentation for the suggested replacement.